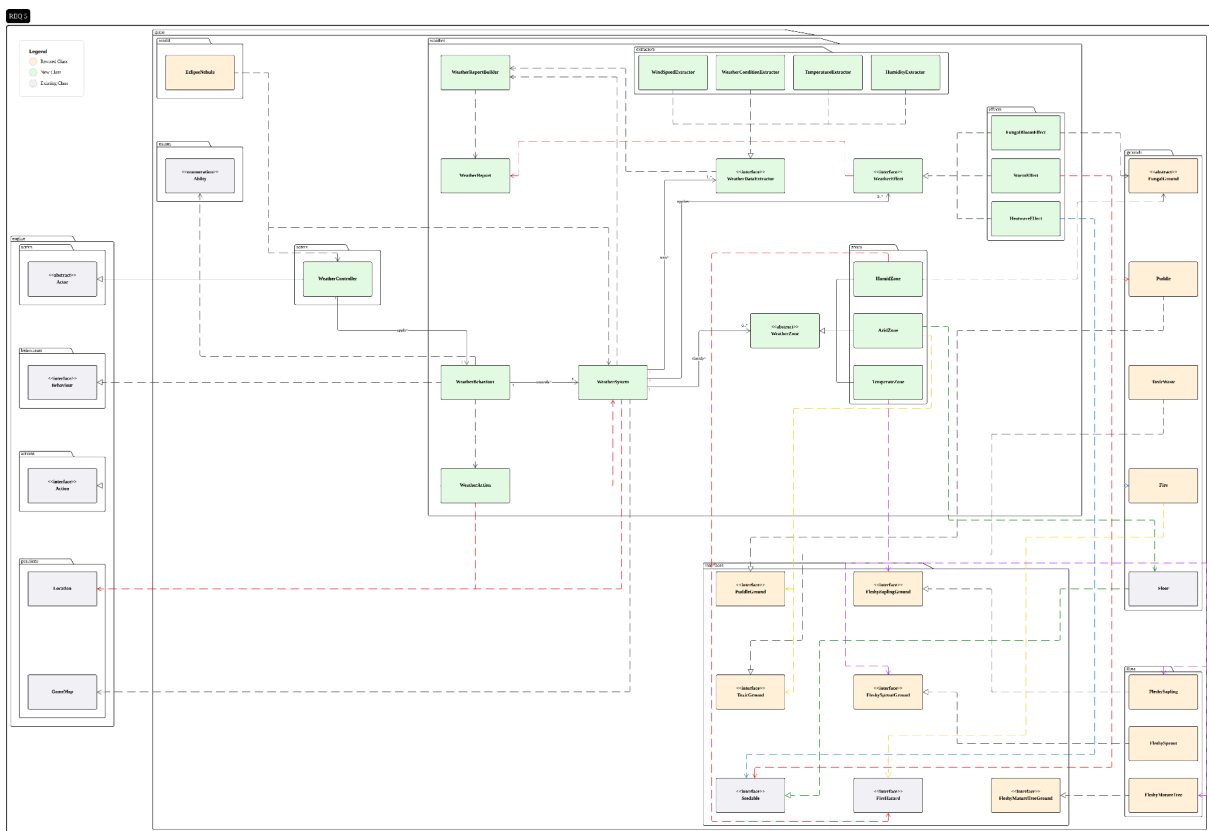


FIT2099 Assignment 3 Design Documentation

Design Diagrams

REQ 4 UML Class Diagram



Design Rationale

1. The Resonator Interface

Design A: Interface-Based Polymorphism

The system uses a Resonator interface to abstract shockwave activation across three distinct item types (TremorBeacon, QuakeCharge, ResonanceMallet). Each item implements this interface and provides its own shockwave behaviour.

Pros	Cons
Keeps shockwave capability separate from the concrete item hierarchy, which reduces coupling.	The interface itself cannot store state, so item-specific data such as cooldown or durability must remain in the concrete classes.

Makes the design easier to extend with additional resonator items.	
Lets shared helper methods live in the interface through the use of default methods.	

Design B: Concrete Inheritance

Create a single abstract ShockwaveItem class with a template method pattern for triggerShockwave.

Pros	Cons
Default implementations of helper methods can be implemented.	Violates Liskov Substitution Principle, ResonanceMallet doesn't necessarily fit the mold of other resonators.
	Forced to inherit unwanted item behaviors (durability from QuakeCharge).
	Any change to the parent affects all resonators, higher coupling.

Design C: Static Utility Methods

Implement shockwave logic entirely as static methods like ShockwaveUtils.triggerShockwave() with no polymorphism.

Pros	Cons
Easier to test and debug since utility methods under one class object.	Violates Single Responsibility Principle as the utility class becomes a monolithic container.
	Callers must explicitly know which item they're triggering out of all the resonator's helper methods.

2. Ground Degradation

Design A: Abstract Class with Progressive Degradation

The system uses an abstract CrackableGround class that enforces a degradation chain: Floor (level 0) → CrackedFloor (level 1) → RubbleField (level 2).

Pros	Cons
Stores common data such as degradation level once instead of repeating it in each class.	The linear hierarchy is less flexible if a future ground type does not fit the same chain/rules.
Allows each ground stage to override only the behaviour that differs.	

Design B: Interface-Based Hierarchy

Define Crackable as an interface; implement Floor, CrackedFloor, and RubbleField independently.

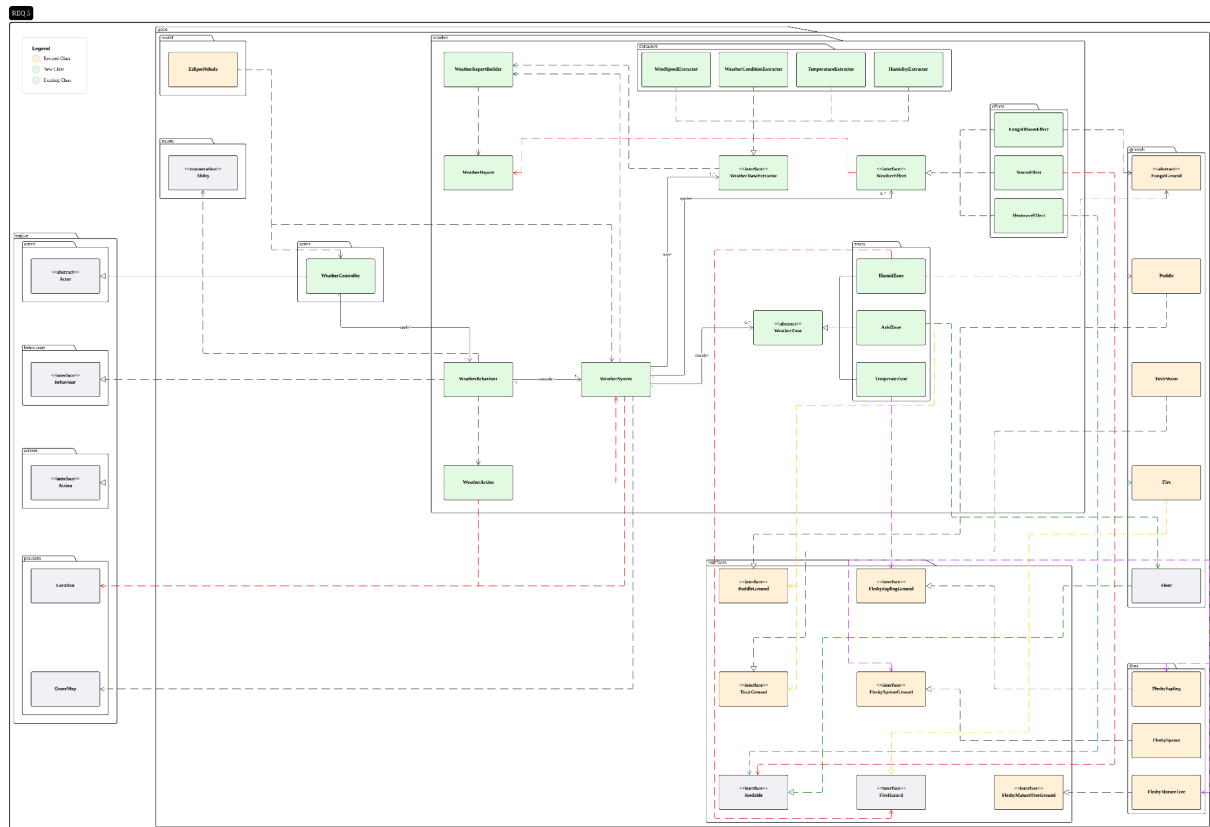
Pros	Cons
Greater flexibility as ground types aren't forced into a linear chain	Must check instance of to determine if a ground is crackable, increasing coupling.
	More difficult to extend where each new degradation stage requires understanding the full pattern.

Design C: Single Monolithic Class with State Enum

Create one 'DegradableFloor' class that tracks its current state (FLOOR, CRACKED, RUBBLE) and behaviour changes based on that state.

Pros	Cons
No inheritance hierarchy, simpler to understand.	Violates Single Responsibility Principle. One class handles three distinct behavioural states (Rubble can fill holes but others cannot).
	More difficult to extend where each new degradation stage requires understanding the full pattern.
	Adding a fourth stage requires modifying the enum and all state-checking logic.

Conclusion



The diagram above shows the final design of Requirement 4, which utilises Design A in Part 1 and Design A in Part 2.

The final design uses the Resonator interface to abstract shockwave activation across the three resonator items: TremorBeacon, QuakeCharge, and ResonanceMallet. Each item implements the same contract while providing its own activation method, radius, and behaviour. This approach promotes polymorphism and reduces coupling between the shockwave system and the concrete item implementations. Instead of depending on specific resonator types, other parts of the system depend only on the Resonator abstraction, aligning with the Dependency Inversion Principle. The design also follows the Open/Closed Principle because new resonator items can be introduced by implementing the Resonator interface without requiring modifications to existing shockwave logic. Using an interface rather than a common abstract item class avoids forcing unrelated items into the same inheritance hierarchy, allowing each resonator to maintain only the state and behaviour relevant to itself. This improves flexibility while avoiding unnecessary coupling that would arise from a shared parent class.

The terrain degradation system is implemented using the abstract class CrackableGround, which provides a common abstraction for all degradable terrain states including Floor, CrackedFloor, and RubbleField. Since all terrain states share common concepts such as degradation level and degradation behaviour, placing this shared functionality within a common abstract class reduces code duplication and adheres to the DRY principle. Each concrete ground type is responsible only for implementing the behaviour that differs from the other stages, such as collapse logic, grace period handling, or hole-filling behaviour. This separation of responsibilities improves maintainability and

aligns with the Single Responsibility Principle. Furthermore, the progressive degradation hierarchy supports the Open/Closed Principle because additional degradation stages can be introduced by extending CrackableGround without modifying existing terrain classes.

The interaction between the resonator system and the terrain degradation system further demonstrates good object-oriented design. Resonators are responsible only for generating shockwaves, while individual ground classes determine how they respond to those shockwaves and subsequent actor interactions. This separation prevents the resonator classes from containing terrain-specific logic and prevents terrain classes from needing knowledge of individual resonator implementations. By distributing responsibilities to the most appropriate abstractions, the design achieves high cohesion and low coupling, making the system easier to extend and maintain as additional resonators, terrain types, or degradation mechanics are introduced in future iterations.